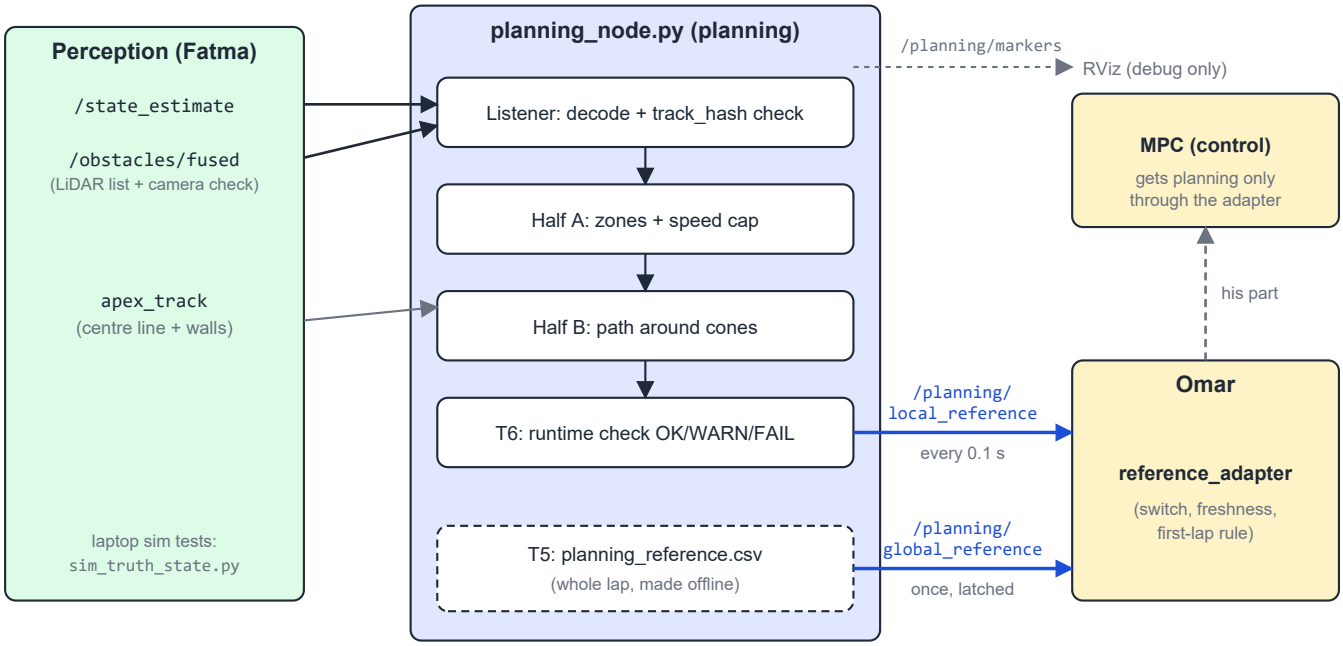
Planning to reference_adapter: ReferenceTrajectory v0.1

From the planning team to Omar (integration / simulator) · 8 November 2026

1. Where planning stands

- **Planning already works with Fatma's data.** Our node subscribes to her `/state_estimate` and `/obstacles/fused` (her obstacle contract v0.2: the LiDAR obstacle list plus her camera check), reads every field correctly and drops messages made for another track (track_hash check). It ran live in ROS 2 Humble with the F1TENTH simulator, her obstacle node, fake camera and fusion node: the car's state and the cones come in, and the node makes a plan every 0.1 s.
- **Our output does not go to the MPC yet.** Planning will not talk to the MPC directly: everything we produce goes to your reference_adapter, which decides what the MPC gets.
- **What we send you:** one message type, **ReferenceTrajectory**, on two topics: **GLOBAL** (the whole racing line around the lap) and **LOCAL** (the next 8 m, including the path around cones). Both are built and tested on our side.

2. How the nodes connect



Solid arrows = built and tested. Blue = what we send you. Dashed = not ours (your adapter feeding the MPC) or debug only.

3. The message: ReferenceTrajectory

One type for both references. The `source` field says which one it is. Units and signs are the same as our planning_reference.csv. Every array has one entry per point, and all arrays have the same length.

Field	Type	Meaning
`header`	std_msgs/Header	stamp = when the reference was made, frame_id = `map`
`source`	uint8	`SOURCE_GLOBAL = 0` (whole lap) or `SOURCE_LOCAL = 1` (8 m ahead)
`track_id`	string	track name, e.g. `closed1`
`track_hash`	string	fingerprint of Fatma's centre line it was made for: reject it if it differs from the track you loaded
`boundary_hash`	string	fingerprint of Fatma's walls it was made for
`lap_length_m`	float64	lap length [m]
`valid`	bool	false = do not drive it

Field	Type	Meaning
`reason`	string	why it is not valid ("" when valid)
`version`	uint32	+1 for every new message
`s_m[]`	float64[]	distance along the centre line [m]. GLOBAL: 0 .. lap length. LOCAL: the car's running s (keeps growing past the lap end)
`x_m[], y_m[]`	float64[]	the same points as map x, y [m]
`e_y_ref_m[]`	float64[]	where to be: sideways distance from the centre line [m], + = left
`e_psi_ref_rad[]`	float64[]	which way to point: heading minus centre-line heading [rad], + = left
`kappa_ref_1pm[]`	float64[]	how much to turn: curvature [1/m], + = left
`v_ref_mps[]`	float64[]	how fast [m/s]
`e_y_min_m[]`	float64[]	corridor for the car's centre, right edge [m] (negative)
`e_y_max_m[]`	float64[]	corridor for the car's centre, left edge [m]

The corridor is not the walls. It is how far the car's *centre* may go: Fatma's walls minus 0.25 m (half the car's width 0.15 m + a 0.10 m buffer), with the tight inside corners trimmed. There is no acceleration field yet, because its meaning still has to be agreed with control.

4. The two topics

	GLOBAL	LOCAL
Topic	`/planning/global_reference`	`/planning/local_reference`
What	our racing line around the whole lap (T5)	the next 8 m ahead of the car (T7): the racing line when the track is clear, the path around the cones when there are cones
Points	every 5 cm, whole lap (closed1: 946 points)	every 0.25 m, 8 m ahead = 33 points
Sent	once at start-up	every 0.1 s (10 Hz), the MPC's cycle
QoS	reliable, **transient local**, depth 1 (latched: a late subscriber still gets it)	reliable, volatile, depth 10
`valid`	true when our offline checks passed	true unless our runtime check (T6) says FAIL; `reason` lists the T6 reasons

LOCAL rules:

- A cycle we skip (no car state yet, or the newest state is older than 50 ms) is still sent: `valid = false`, no points, and `reason` says why (e.g. "waiting for the car"). So you can tell "planning is alive but has no plan" apart from "planning stopped".
- `version` goes up by 1 with every message.
- `s_m` keeps growing past the lap end (no jump back to 0), so it always increases along the message. Use `lap_length_m` to wrap it if you need 0 .. lap length.

Example from our live test (closed1, a test car driving the racing line at 2 m/s):

	Received
GLOBAL	946 points, s 0 .. 47.24 m, lap 47.29 m, v 4.88 .. 6.00 m/s, valid, track_hash `15f62f337c4c8787`, boundary_hash `2e17a89df3452c0e`
LOCAL	60 messages in 6 s, version +1 each time. Before the car existed: valid false, 0 points, "waiting for the car". Then: valid, 33 points, s 19.80 .. 27.80 m with the car at 20 m, e_y -0.27 .. +0.45 m. v was 3.08 m/s because no obstacle messages ran in that test, and our planner caps the speed when it cannot trust the obstacle list.

5. Track data (centre line and walls)

The track geometry is Fatma's, not ours: take it from her **apex_track** package (the track's `track.yaml`), the same file our planner loads. Check its track_hash and boundary_hash against the ones in our messages to be sure we use the same track. You need it to turn s / e_y into map positions (`track.point(s)`), though our messages already carry `x_m` / `y_m` for every point.

6. How to build and run

Our message package (only needed until ReferenceTrajectory is in apex_interfaces): `ros/apex_planning_msgs` in our repo, a normal ament_cmake message package. One way to build it:

```
mkdir -p ~/planning_ws/src
ln -s <our repo>/ros/apex_planning_msgs ~/planning_ws/src/apex_planning_msgs
cd ~/planning_ws && colcon build
source ~/planning_ws/install/local_setup.bash
```

Then in Python: `from apex_planning_msgs.msg import ReferenceTrajectory`

Fatma's workspace must be her latest version (apex_msgs 0.4.0, with the camera fields) and built with `colcon build --symlink-install`: our node finds her track files through that link, and with a plain build it cannot load the track.

Run the planning node from our repo folder, with ROS 2 Humble, Fatma's workspace (apex_msgs, apex_track) and our message package sourced. Her obstacle node **and her fusion node** (`ros2 launch sensor_fusion fusion.launch.py`) must be running: without the fusion node `/obstacles/fused` stays silent and our planner slows the car down (it cannot trust the obstacle list).

```
python3 planning_node.py
```

It sends GLOBAL at start-up, then LOCAL every 0.1 s. To check that it is working:

```
ros2 topic hz /planning/local_reference      # about 10 Hz
ros2 topic echo --once --qos-durability transient_local \
     /planning/global_reference --field track_hash
```

7. Warnings

- **CPU:** on a 4-core laptop (i5-8265U), Fatma's full localization plus the simulator did not keep up and the state went LOST. Your noVNC setup needs enough cores for both.
- **Test stand-in, not a truth adapter:** for laptop tests we use `sim_truth_state.py`, which turns the simulator's odometry into `/state_estimate` live, so we don't need her localization. Your truth_adapter is for recordings only. The two are different things; please don't mix them up.

8. Questions for you

1. Should ReferenceTrajectory go into your **apex_interfaces**, or will you use our `apex_planning_msgs`? If it goes into yours, we only change one setting on our side.

2. Is it OK for your adapter that LOCAL's `s_m` keeps growing past the lap end, and that skipped cycles are sent as invalid messages with no points? Or would you like either one done differently?

3. A proposal to agree on: our node **always** sends LOCAL (the racing line when the track is clear) with its validity from T6. Your adapter only switches between LOCAL and GLOBAL based on validity, freshness and the first-lap rule. Does that fit how you planned the adapter?

Attached

- `ReferenceTrajectory.msg`: the message definition
- `apex_planning_msgs/`: our message package (package.xml, CMakeLists.txt, msg/)